SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis | 979-8-3503-5554-3/24/$31.00 ©2024 IEEE | DOI: 10.1109/SCW63240.2024.00074

ProTEA: Programmable Transformer Encoder Acceleration on FPGA

Ehsan Kabir*, Jason D. Bakos†, David Andrews*, Miaoqing Huang*
*Department of Electrical Engineering and Computer Science, University of Arkansas, Fayetteville, USA
†Department of Computer Science and Engineering, University of South Carolina, USA
{ekabir, dandrews, mqhuang}@uark.edu, jbakos@cse.sc.edu

***Abstract*—Transformer neural networks (TNN) have been widely utilized on a diverse range of applications, including natural language processing (NLP), machine translation, and computer vision (CV). Their widespread adoption has been primarily driven by the exceptional performance of their multi-head self-attention block used to extract key features from sequential data. The multi-head self-attention block is followed by feedforward neural networks, which play a crucial role in introducing non-linearity to assist the model in learning complex patterns. Despite the popularity of TNNs, there has been limited numbers of hardware accelerators targeting these two critical blocks. Most prior works have concentrated on sparse architectures that are not flexible for popular TNN variants. This paper introduces *ProTEA*, a runtime programmable accelerator tailored for the dense computations of most of state-of-the-art transformer encoders. *ProTEA* is designed to reduce latency by maximizing parallelism. We introduce an efficient tiling of large matrices that can distribute memory and computing resources across different hardware components within the FPGA. We provide run time evaluations of *ProTEA* on a Xilinx Alveo U55C high-performance data center accelerator card. Experimental results demonstrate that *ProTEA* can host a wide range of popular transformer networks and achieve near optimal performance with a tile size of 64 in the multi-head self-attention block and 6 in the feedforward networks block when configured with 8 parallel attention heads, 12 layers, and an embedding dimension of 768 on the U55C. Comparative results are provided showing *ProTEA* is 2.5× faster than an NVIDIA Titan XP GPU. Results also show that it achieves 1.3 – 2.8× speed up compared with current state-of-the-art custom designed FPGA accelerators.**

***Index Terms*—FPGA, Transformer, Attention, Neural Networks, Encoder, High-Level Synthesis, Natural Language Processing, Hardware Accelerators.**

I. Introduction

In recent years, transformer neural networks have become widely utilized on a diverse range of applications including natural language processing (NLP) [1], [2], neural machine translation [3], and image processing [4]. They are becoming favored over traditional recurrent neural network (RNN) and long short-term memory (LSTM) models for NLP tasks, and convolutional neural networks (CNN) for CV tasks. Their popularity is being driven by their ability to enable high computational parallelism for both the training and inference steps. Their natural exposure of higher levels of parallelism makes them well-suited for acceleration on hardware such as GPUs and FPGAs. There exist many transformer-based models such as full transformers containing both encoder and decoder [2], BERT [5], RoBERTa [6], Swin Transformers [7], structBERT [8] etc. These models incorporate two notable features: a multi-headed attention (MHA) mechanism and feedforward neural networks (FFN) that distinguishes them from traditional CNNs, RNNs, and LSTMs. These MHA and FFN mechanisms are computationally expensive due to intensive matrix-matrix multiplications and complex data flows [9]. They account for a significant portion of runtime in many existing TNNs [10]. Unfortunately, executing TNNs is inefficient on general-purpose platforms such as GPUs and CPUs because of their high power consumption, low computational efficiency, underutilized memory bandwidth, and significant compilation overheads [11]. In addition to GPUs, FPGAs have become popular commercial off the shelf components used to accelerate DNNs. FPGAs offer the ability to exploit high level of parallelism to provide low run time inference latencies with efficient power consumption [12], [13]. Many studies have investigated how to increase the parallelization of CNNs, LSTMs, Graph Convolutional Networks [14]–[17] on FPGAs to enhance performance. Recently, TNNs have been successfully deployed on FPGAs and application-specific integrated circuit (ASIC) hardware accelerators [18]–[20]. Most implementations compress the model by using different weight pruning strategies, and reduce latency by incorporating sparse matrices. Thus, they use a specialized sparse architecture specific to each application. However, different applications require different sparsity patterns, necessitating the redesign of the hardware architecture for optimal results. This comes at the cost of time-consuming synthesis, and requires skills in digital design and computer architecture as well as detailed knowledge of each target logic family. Therefore, there is a need for a versatile accelerator capable of efficiently managing dense matrix computations across a range of TNN applications.

The study in [18] uses logic resources to implement a systolic array for parallelism, which can lead to underutilization of digital signal processing (DSP) units that are capable of high-speed computation at higher frequencies. DSP utilization also depends on the implementation method. For instance, many accelerators [20]–[23] employ high-level synthesis (HLS) tools, while others use hardware description language (HDL) [24]–[26] for design. Although HLS requires less implementation time compared to HDL, writing efficient HLS code that effectively manages specific FPGA resources, such as DSPs, for optimal performance remains challenging

979-8-3503-5554-3/24/$31.00 ©2024 IEEE
DOI 10.1109/SCW63240.2024.00074

[15].

The analysis in [27]–[30] demonstrated that MHA and FFN occupy major portions of the memory and they have the highest computational demands. Since on-chip memory of FPGAs typically does not exceed 36MB and off-chip memory bandwidth is sometimes limited, matrices must be partitioned into tiles. However, designing an optimal partitioning scheme for MHA and FFN that aligns effectively with the architecture presents a significant challenge.

In this paper, HLS tool was used to design ***ProTEA***, a programmable accelerator for transformer encoders. The code of the design written in HLS was optimized to increase the parallel computations by the DSPs. ***ProTEA*** incorporates efficient tiling for both the attention mechanism and linear transformations. It ensures enhanced parallel computations and communication so that the transformer encoding can be accelerated as much as possible.

The contributions of this paper are:

- A novel accelerator architecture for transformer encoders that maximizes DSP utilization to enhance parallel processing and achieve low latency.
- An efficient tiling strategy for weight matrices in both the multi-head attention layer and the feedforward neural network layer, enabling the accommodation of large models within on-chip memory.
- A parameterized HLS code that allows for design-time adjustments of parameters in the HLS tool.
- A runtime programmable feature enabling dynamic adjustment of parameters in software, facilitating the evaluation of different models without the need for hardware re-synthesis.

II. Background

Transformers consist of several fundamental components, as depicted in Fig. 1. An input sequence of tokens is first converted into embeddings. The positional encoder adds positional information to these embeddings, enabling the model to account for the order of tokens in a sequence. This encoder generates vectors that provide context based on each word's position in a sentence. These vectors are then linearly transformed into three tensors: Q (queries), K (keys), and V (values) by multiplying the embedding matrix with three distinct weight matrices. The encoder block processes these tensors, transforming them into a higher-level representation that captures essential information. This transformation is crucial for accurately capturing features and contextual relationships within the input sequence. The encoder architecture is composed of two primary sub-layers: (1) the self-attention mechanism, and (2) the position-wise feed-forward network.

The self-attention mechanism allows the model to simultaneously evaluate different parts of an input sequence, capturing long-range relationships by calculating attention scores and using multi-head projections for various input representations. This capability enables the model to effectively learn complex patterns, dependencies, and relationships. The position-wise feed-forward network (FFN), similar to a multilayer perceptron (MLP), applies linear transformations independently to each position in the input sequence. This network performs two linear transformations, primarily involving matrix-vector multiplication. The first transformation includes activation functions such as the Rectified Linear Unit (ReLU) or Gaussian Error Linear Unit (GeLU), while the second transformation does not.

Additionally, each sub-layer incorporates a residual connection combined with layer normalization (LN), addressing the vanishing gradient problem during training. Residual connections and LN layers are added after each MHA and FFN layer, involving the addition of matrix elements and nonlinear functions.

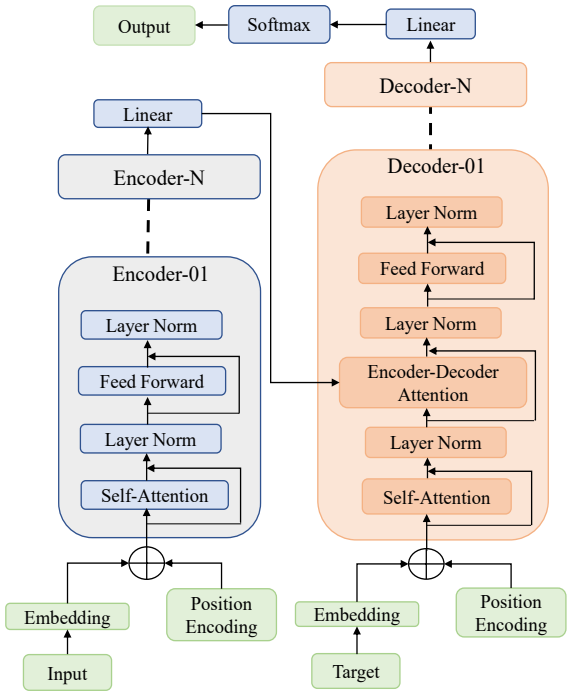

Fig. 1. Transformer Architecture.

The decoder block, depicted in Fig. 1, is tasked with generating the output sequence using the encoded representations provided by the encoder. Similar to the encoder, the decoder comprises a stack of N identical layers. Each layer in the decoder includes three sub-layers: (1) the Masked Attention Mechanism, which is similar to the encoder's self-attention but incorporates a masking feature to prevent the output from depending on future outputs; (2) an attention layer that focuses on the encoder's output, allowing the decoder to highlight relevant parts of the input sequence for each output element; and (3) a position-wise feed-forward network.

As shown in Fig 2, the scaled dot-product attention in each head is a vital component of the multi-head attention layer. The attention weights are calculated by taking the dot product of

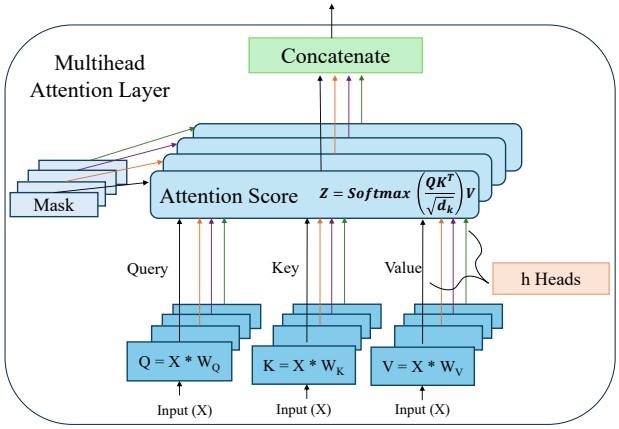

Fig. 2. Multihead Attention Layer.

the Q and K matrices and then scaling the result by the square root of the second dimension of the K matrix. This scaling is crucial to prevent the dot products from becoming too large, which helps stabilize gradients during training. The scaled dot products are then passed through the softmax function to compute the attention weights. These weights are used to perform a weighted sum of the value vectors. The final output is the projection of the concatenated sequences from all heads.

The output of MHA can be represented as Equation 1 & 2. The input sequence X is linearly mapped into Q_i, K_i, V_i matrices using weights and biases. The parameter $d_k = d_{model}/h$ is the 2nd dimension of Q_i and K_i. d_{model} is a hyperparameter called embedding dimension and h is number of heads. 'i' is the index for attention heads.

$$Attention(Q_i, K_i, V_i) = softmax\left(Mask\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right)\right)V_i \tag{1}$$

$$\begin{aligned} Q_i &= X \times W_q + B_q, \\ K_i &= X \times W_k + B_k, \\ V_i &= X \times W_v + B_v \end{aligned} \tag{2}$$

III. Related Work

Various FPGA and ASIC accelerators have been designed for TNNs. The ASIC design in [19] leveraged parallelism and specialized datapaths to achieve significant gains in performance and energy efficiency. Another ASIC, ELSA [10], employed specialized approximation algorithms to reduce computational demands. The SpAtten [31] ASIC utilized sparsity and quantization to decrease computations and memory access. Additionally, the hardware-software co-design framework Sanger [9] facilitated dynamic sparsity through a reconfigurable ASIC architecture. Despite these advancements, these solutions primarily focus on accelerating sparse attention mechanisms and do not address the deployment of full transformer models. The FPGA accelerator proposed by Lu et al. [18] is the first hardware architecture to accelerate both the MHA and FFN layers of the transformer. However, their implementation was done using HDL for a single attention head. A shared computing architecture is implemented in [32], where a parallel computing array is shared between MHA and FFNs for a CNN application. A novel structural pruning method was proposed by [33] and the associated accelerator on FPGA was designed to reduce memory footprint. Peng et al. [21] explored column-balanced block-wise pruning for transformers and designed an FPGA accelerator for optimized block-wise matrix multiplication. An algorithm hardware framework [28] utilizes latency and accuracy constraints to determine the optimal sparsity ratio and select an appropriate FPGA platform. The energy-efficient acceleration framework FTRANS [29] features an improved block-circulant matrix method for algorithm-level sparsity, along with a custom-designed accelerator tailored for this approach. Wojcicki et al. [23] deployed a small TNN model on FPGA using HLS for experiments at the Large Hadron Collider. All of the existing hardware architectures are designed for a specific TNN and a specific sparsity pattern. They lack the flexibility to reconfigure the computing structure for different applications during runtime. EFA-Trans [25] is compatible with dense and sparse computing patterns, but it would need resynthesis of the hardware to switch between two options. Furthermore, none of them explored which tile size and what utilization DSPs could achieve optimum parallelism.

IV. Accelerator Architecture

The core of the accelerator is designed in C language on Vitis HLS 2022.2.1 tool. C simulation verifies the correctness of the algorithm, while C/RTL co-simulation ensures the functionality of the synthesized hardware. This section describes the high-level synthesis design technique that generates an optimized architecture utilizing most of the DSPs in the computation engines, ensuring high parallelism. The overall structure of the accelerator contains two main processing modules - the multihead attention (MHA) module and the feedforward network (FFN) module, which are shown in Fig. 3 and Fig. 4 respectively. The overall system was designed in Vivado 2022.1.2 design suite. It contains a custom IP block for the accelerator, which is exported from HLS. The inputs and weights are fetched from off-chip high-bandwidth memory (HBM) using AXI4 master interfaces [34] when the load instruction from the accelerator controller is received according to demand. The accelerator receives control signals from the processor through an AXI-lite slave interface [35]. Each hyperparameters of TNN can be programmed during runtime up to a maximum value by MicroBlaze (μB) softcore processor [36].

A. Attention Module

The attention module (Fig. 3) comprises three computation engines (CE), labeled as QKV_{CE}, QK_{CE}, and SV_{CE} based on their outputs. The number of these engines is determined

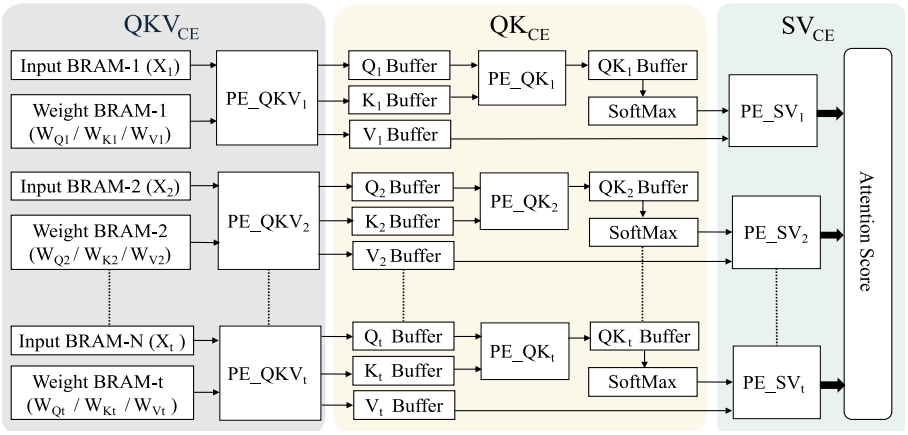

Fig. 3. Computations of the Attention Module

by the number of attention heads (h). Each engine features an array of processing elements (PE), where each PE includes a DSP48 for performing multiplication and accumulation (MAC) operations. The quantity of PEs denoted as 't' is influenced by the unrolling factor of the inner loop and the initiation interval of the pipelined outer loop. Since the data access patterns and computational needs vary across different engines, each has separate function definition in HLS. This ensures that the synthesized RTL modules of the engines contain distinct PE arrays, enabling individual optimization. Input data and weights are stored in multiple BRAMs/LUTRAMs to support parallel access.

Each PE operates independently, equipped with its own memories, controller, and computing units. In HLS, the weights (W_q, W_k, W_v) for generating the query (Q), key (K), and value (V) matrices are defined as separate two-dimensional arrays of size ($\frac{d_{model}}{h} \times TS_{MHA}$). Here, TS_{MHA} represents the tile size in the attention module. It is the dimension of the sub-matrices into which the larger weight matrices are partitioned. The number of heads, tile size, and array partitioning directives in HLS determine how these arrays are divided to create multiple two-port BRAMs. To address the limited ports of BRAMs, array partitioning and data loading are optimized to ensure that data needed simultaneously by a DSP is stored in separate BRAMs. The Q, K, and V matrices, sized ($SL \times \frac{d_{model}}{h}$), are stored in intermediate buffers. Here, SL stands for sequence length.

1) ***QKV_{CE} engine:*** QKV_{CE} engine generates the query, key, and value matrices. This engine contains the W_q, W_k, W_v buffers, and input (X_i) buffers from which data is accessed in parallel by parallel DSP units. The arrays used in this engine are divided into subarrays using our tiling technique to fit into on-chip memories. The number of loop iterations in the QKV_{CE} engine is determined by TS_{MHA}, resulting in a total of ($\frac{d_{model}}{TS_{MHA}}$) tiles or iterations. During each iteration, distinct data is loaded into the W_q, W_k, W_v, and X_i buffers. Computations then commence in the PEs, while biases for the Q, K, and V matrices are simultaneously loaded into registers from off-chip memory. These biases are subsequently added to the Q, K, and V matrices. Algorithm 1 illustrates the computations of this engine, where the second loop (line 6) is pipelined, resulting in the full unrolling of the innermost loop (line 8) and generating ($\frac{d_{model}}{TS_{MHA}}$) PEs.

2) ***QK_{CE} engine:*** The QK_{CE} engine performs matrix-matrix multiplication between the Q and K matrices. Since these matrices are relatively small, they are not tiled. Algorithm 2 outlines the operations performed.

Algorithm 1 Q, K, V Calculation

```
1: for i ← 1 to Sequence Length do
2:     #pragma HLS pipeline off
3:     S_q ← 0
4:     S_k ← 0
5:     S_v ← 0
6:     for k ← 1 to Embedding Dimension / Number of Heads do
7:         #pragma HLS pipeline II = 1
8:         for j ← 1 to Tiles in MHA do
9:             S_q ← S_q + x[i][j] × w_q[k][j];
10:            S_q ← S_q + x[i][j] × w_k[k][j];
11:            S_q ← S_q + x[i][j] × w_v[k][j];
12:        end for
13:        Q[i][k] ← Q[i][k] + S_q;
14:        K[i][k] ← K[i][k] + S_k;
15:        V[i][k] ← V[i][k] + S_v;
16:    end for
17: end for
```

The innermost loop (line 6) is fully unrolled, resulting in ($\frac{d_{model}}{h}$) PEs for this engine. The engine generates a matrix (S) of attention weights, which is stored in either BRAM or registers. These values are then passed to the softmax function. The softmax function, implemented in HLS, utilizes LUTs and flip-flops (FFs) to compute the result.

Algorithm 2 $Q \times K^T$ Calculation

```
1: for i ← 1 to Sequence Length do
2:     #pragma HLS pipeline off
3:     for j ← 1 to Sequence Length do
4:         #pragma HLS pipeline II = 1
5:         S ← 0
6:         for k ← 1 to Embedding Dimension / Number of Heads do
7:             S ← S + Q[i][k] × K[j][k];
8:         end for
9:         s[i][j] ← S/Embedding_Dimension;
10:    end for
11: end for
```

3) ***SV_{CE} engine:*** The output matrix (S) from the softmax operation is passed to the SV_{CE} engine (Algorithm 3), where it undergoes matrix-matrix multiplication with the value (V) matrix. In Algorithm 3, the innermost loop (line 6) is fully unrolled, resulting in (SL) PEs. The output from this engine is termed the attention score.

Algorithm 3 $S \times V$ Calculation

```
1: for i ← 1 to Sequence Length do
2:     #pragma HLS pipeline off
3:     for j ← 1 to Embedding Dimension / Number of Heads do
4:         #pragma HLS pipeline II = 1
5:         vv ← 0
6:         for k ← 1 to Sequence Length do
7:             vv ← vv + S[i][k] × V[k][j];
8:         end for
9:         SV[i][j] ← vv;
10:    end for
11: end for
```

B. Feedforward Network Module

There are three CEs, denoted as $FFN1_{CE}$, $FFN2_{CE}$, and $FFN3_{CE}$ in FFN to perform the operations of feedforward networks of different dimensions (Fig. 4). The definitions of the functions representing the CEs have different dimensions of arrays for the inputs and outputs in HLS. These arrays are converted into BRAMs/LUTRAMs after synthesis. The number of computations inside each engine is different, which is why each has a separate function in HLS. They contain a different number of processing elements (PE) after synthesis because of different unrolling factors of the innermost loop. The weights are stored in a two-dimensional array (W_o) of size ($\frac{d_{model}}{TS_{FFN}} \times \frac{(4 \times d_{model})}{TS_{FFN}}$) in HLS, where TS_{FFN} is tile size in FFN. $FFN1_{CE}$ and $FFN3_{CE}$ are followed by layer normalization (LN) modules. Algorithm 4 describes the general coding approach for an FFN engine.

1) ***$FFN1_{CE}$ engine:*** $FFN1_{CE}$ engine performs the first linear transformation on the attention scores. The arrays used by the PEs are tiled along both dimensions. Thus, this engine is accessed $TS_{FFN} \times TS_{FFN}$ times to finish the complete operation. The second for loop of the HLS code is pipelined causing the innermost for loop to be fully unrolled. This generates TS_{FFN} PEs which equals to $\frac{d_{model}}{Tile\ no.\ FFN}$.

2) ***$FFN2_{CE}$ engine:*** $FFN2_{CE}$ engine performs second linear transformation on the normalized outputs of $FFN1_{CE}$ engine. The arrays used by the PEs are tiled along both dimensions. Thus, this engine is accessed $4 \times TS_{FFN} \times TS_{FFN}$ times to finish the complete operation. This engine also contains TS_{FFN} PEs which equals to $\frac{d_{model}}{Tile\ no.\ FFN}$, because the trip count of the innermost loop is $\frac{d_{model}}{Tile\ no.\ FFN}$ and it is fully unrolled.

3) ***$FFN3_{CE}$ engine:*** $FFN3_{CE}$ engine performs final linear transformation on the normalized outputs of $FFN2_{CE}$ engine. The arrays used by the PEs are tiled along both dimensions. Thus, this engine is accessed $4 \times TS_{FFN} \times TS_{FFN}$ times to finish the complete operation. The complete unroll of the innermost loop generates $4 \times TS_{FFN}$ PEs in it, which equals to $\frac{4 \times d_{model}}{Tile\ no.\ FFN}$.

Algorithm 4 FFN Computation Example

```
1: for i ← 1 to Sequence Length do
2:     #pragma HLS pipeline off
3:     m ← index × Embedding Dimension / Tile no. FFN
4:     for j ← 1 to Embedding Dimension / Tile no. FFN do
5:         #pragma HLS pipeline II = 1
6:         sum ← 0
7:         for k ← 1 to Embedding Dimension / Tile no. FFN do
8:             sum ← sum + inputs[i][k] × weights[k][j];
9:         end for
10:        output[i][m] ← output[i][j] + sum;
11:        m ← m + 1;
12:    end for
13: end for
```

C. Tiling Technique

Since transformer models are typically large, tiling is used to manage the utilization of on-chip memory and computing units effectively. It ensures that the HLS tool can efficiently partition arrays and pipeline or unroll loops to minimize latency while keeping compilation time short. Figure 5 illustrates our distinctive tiling strategy for the MHA module. The weight matrices are divided into tiles, enabling BRAMs to be loaded with partial data fetched from off-chip memory. Tiling is applied only along the second dimension (columns) of the matrix because the first dimension (rows) is already reduced by the number of heads. Consequently, each matrix is loaded ($\frac{d_{model}}{TS_{MHA}}$) times. The input buffers for each attention head are defined as a two-dimensional array of size (SL $\times$ TS_{MHA}), and tiling is similarly applied along the column of the matrix, resulting in ($\frac{d_{model}}{TS_{MHA}}$) loads. During each iteration, data for one tile is loaded initially. The PEs then compute on this data, storing the results in intermediate buffers, which are accumulated with results from previous iterations. Ultimately, the final output is the cumulative sum of the results computed across all tiles.

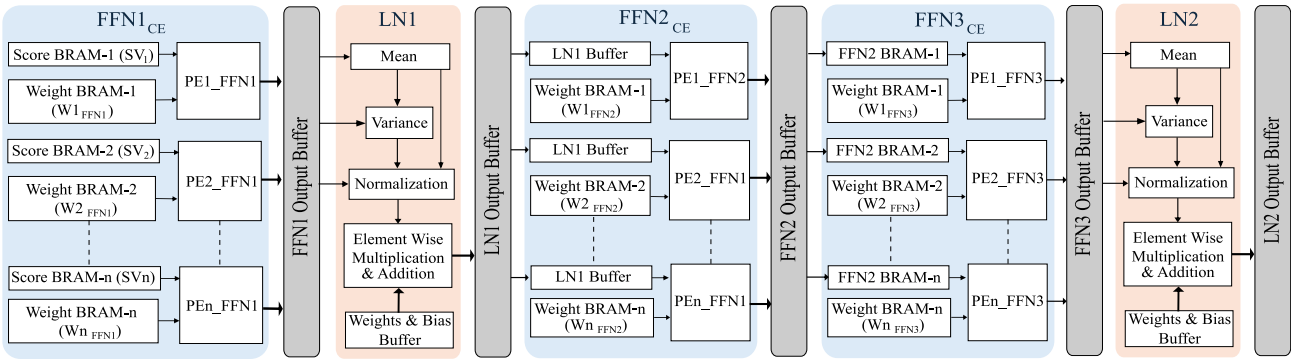

Fig. 4. Computations of Feedforward Network.

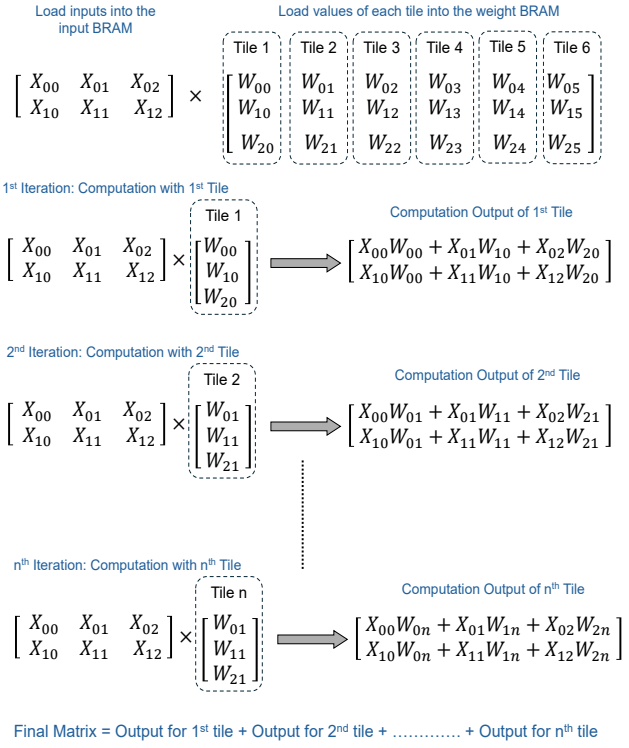

Fig. 5. Tiling Technique in Multihead Attention Layer.

The FFNs that follow the attention layer are the most time- and resource-intensive components. The weight matrices for the FFN are defined as two-dimensional arrays with dimensions $(TS_{FFN}) \times (4 \times TS_{FFN})$. These matrices are tiled along both dimensions (rows and columns), requiring two loops to iteratively load each tile. The first FFN module is reused $(\frac{d_{model}}{TS_{FFN}})^2$ times because both loops iterate $\frac{d_{model}}{TS_{FFN}}$ times. The second and third FFN modules are reused $(\frac{4\times(d_{model})^2}{(TS_{FFN})^2})$ times, reflecting the iteration counts of either $\frac{d_{model}}{TS_{FFN}}$ or $\frac{4\times d_{model}}{TS_{FFN}}$. Figure 6 illustrates our specific tiling strategy for the FFN. Results are first accumulated along the columns, followed by accumulation along the rows for all tiles.

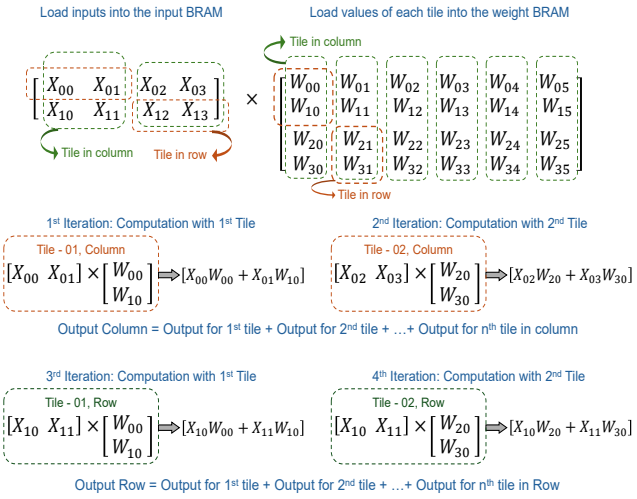

Fig. 6. Tiling Technique in FFN.

D. Runtime Configurable Capability

The runtime-programmable parameters such as the number of attention heads, number of layers, embedding dimension, and sequence length can be sent to ***ProTEA*** via software running on the μB processor. TNN models are trained using the PyTorch framework, and the resulting models should be saved as *'.pth'* files. These files are then processed by a Python interpreter to extract key parameters such as the number of attention heads, layers, embedding dimension, and sequence length. While these parameters will vary across applications, ***ProTEA*** does not require resynthesis for each model; only minor software modifications are necessary. The software, developed in C++ using the Xilinx SDK tool, utilizes the extracted data to generate instructions and control signals. These signals guide the processor in activating the relevant parts of the accelerator hardware.

E. Tile Size Determination

In ***ProTEA***, the programmable parameters can be adjusted at runtime, whereas the tile size must be set before synthesis,

as it cannot be modified without resynthesizing the entire hardware. The graph in Fig. 7 illustrates how variations in TS_{MHA} and TS_{FFN} impact system frequency (MHz) and latency (normalized to the minimum value). The number of tiles in MHA ($\frac{d_{model}}{TS_{MHA}}$) was varied from 6 to 48, and for each MHA tile count, the number of tiles in FFN ($\frac{d_{model}}{TS_{FFN}}$) ranged from 2 to 6. The results indicate that the optimal configuration for achieving the highest frequency (blue color) and lowest latency (green color) was 12 tiles in MHA and 6 tiles in FFN. This setup achieved a maximum frequency of 200 MHz, allowing ***ProTEA*** to execute all transformer neural network models discussed in Section V. Moreover, experiments showed that TS_{MHA} of 64 and TS_{FFN} of 128 are optimal for HLS, allowing for efficient array partitioning within a reasonable compilation time (approximately 36 hours) for a state-of-the-art (SOTA) transformer encoder.

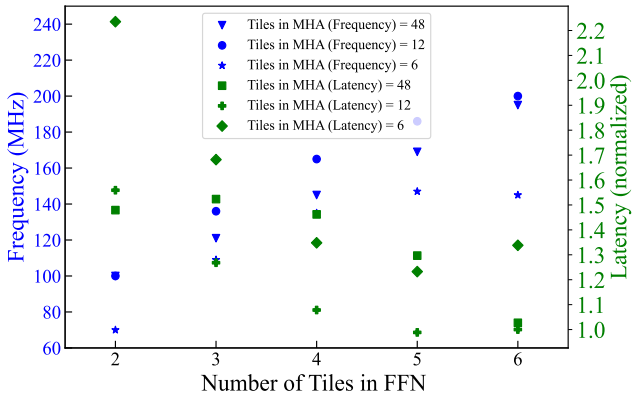

Fig. 7. Choosing the optimum tile size.

V. Evaluation and Results

Table I presents the runtime programmability, resource utilization, and performance metrics of ***ProTEA***. The reported latency reflects the computation time, accounting for the overlap of data loading and computation. The synthesis was conducted with fixed tile sizes of TS_{MHA} = 64 and TS_{FFN} = 128, as these values are set before synthesis and cannot be altered afterward. Data was quantized to 8-bit fixed-point format; while this might result in accuracy loss depending on the application, it was not a primary focus. For applications requiring a larger bit width, the design can be easily modified in the HLS code, which will impact both resource utilization and latency. The accelerator's design parameters, including the embedding dimension (d_{model}), number of heads (h), number of layers (N), and sequence length (SL), were initially configured with fixed values — 768, 8, 12, and 64 respectively — based on a variant of BERT [5] and the available FPGA resources. These parameters were then adjusted dynamically at runtime using μB. This approach allows ***ProTEA*** to be synthesized once for a fixed set of resources while retaining the flexibility to adapt to various architectures as needed.

Tests 1, 2, and 3 demonstrate how varying the number of attention heads within the same accelerator dynamically impacts latency and throughput, with throughput defined as the number of giga operations per second (GOPS). On the Alveo U55C, the lowest latency of 279 ms and the highest GOPS of 53 were achieved with 8 parallel heads. Tests 4 and 5 explore the effect of varying the number of layers, showing that latency decreases and GOPS increases as the number of layers is reduced. Tests 6 and 7 examine the impact of embedding dimensions, with latency increasing and GOPS decreasing as the embedding dimension grows. Finally, Tests 8 and 9 investigate the effect of varying sequence length, where performance deteriorates as sequence length increases.

Resource utilization remained consistent across Tests 1 to 9, as the accelerator was synthesized only once with a fixed tile size, while other parameters were reconfigured at runtime through software. The design achieved high resource utilization, with 40% of DSPs and 76% of LUTs in use. Further DSP utilization was limited by the available LUTs, and the optimal number of parallel attention heads was determined to be 8 on the Alveo U55C to avoid overutilization by the QKV_{CE} engine.

Table II compares the performance of our accelerator, ***ProTEA***, with other FPGA-based accelerators. Each of these accelerators is custom-built for a specific TNN model, with some designed specifically for sparse computations. Among them, only EFA-Trans [25] is flexible enough to toggle the sparse preprocessing unit, allowing it to switch between sparse and dense computations. Since ***ProTEA*** was synthesized only once with a fixed set of hardware resources and bit width, and was implemented on a different platform, we evaluated performance metrics like latency, throughput (GOPS), and normalized throughput (GOPS per DSP) [15] for a fair comparison. ***ProTEA*** achieved 2.8× and 1.7× improvements in speed and GOPS, respectively, compared to the accelerators proposed by Wojcicki et al. [23] and Qi et al. [28]. The GOPS/DSP ratio was also increased by 3.46× and 2× compared to these accelerators. On the other hand, EFA-Trans, which appears to be custom-designed using HDL methods, resulted in more efficient hardware with a lower level of abstraction, making it 3.5× faster than ***ProTEA***. Peng et al. [21] applied a high sparsity of 90% to their model, achieving a 14× speedup over ***ProTEA***. If the same sparsity level were applied to *ProTEA*, its latency would mathematically be reduced to 0.448 ms (calculated as $4.48 - 4.48 \times 0.9$), making it 1.4× slower. FTRANS [29] compressed the model by 93%. The same compression would make ***ProTEA*** 9.4× faster because its latency would be 0.31 ms (calculated as $4.48 - 4.48 \times 0.93$). Moreover, ***ProTEA*** demonstrated 2× higher GOPS/DSP than FTRANS, indicating more efficient DSP usage.

Table III compares ***ProTEA*** with various GPUs and CPUs operating at frequencies between 1.3 and 3.2 GHz. ***ProTEA*** was tested with different TNN models, as referenced in the second column. We could easily adjust the embedding dimensions, number of heads & layers, and sequence length in runtime to align with the architectures in the referenced studies without altering the hardware, thus, ensuring a fair

TABLE I
OVERALL RESULTS FOR OUR ACCELERATOR.

Test no.	Sequence Length	Embedding Dimension	Number of Heads	Number of Layers	Data Format	DSPs	LUTs	FFs	Latency (ms)	GOPS
#1	64	768	8	12	8bit fixed	3612 (40%)	993107 (76%)	704115 (27%)	279	53
#2			4						285	51
#3			2						295	49
#4	64	768	8	8	8bit fixed	3612 (40%)	993107 (76%)	704115 (27%)	186	80
#5				4					93	159
#6	64	512	8	12	8bit fixed	3612 (40%)	993107 (76%)	704115 (27%)	186	36
#7		256							95	18
#8	128	768	8	12	8bit fixed	3612 (40%)	993107 (76%)	704115 (27%)	560	54
#9	32								165	44

TABLE II
COMPARISON WITH FPGA ACCELERATORS.

Accelerator	Precision	FPGA	DSP	Latency (ms)	GOPS	(GOPS/DSP)× 1000	Method	Sparsity
[21]	–	Alveo U200	3368	0.32	555	164	HLS	90%
ProTEA	Fix8	Alveo U55C	3612	4.48	79	22		0%
[23]	Float32	Alveo 250	4351	1.2	0.0006	0.00013	HLS	0%
ProTEA	Fix8	Alveo U55C	3612	0.425	0.0017	0.00045		
[25]	Int8	ZCU102	1024	1.47	279	272	HDL	0%
ProTEA	Fix8	Alveo U55C	3612	5.18	83	23	HLS	
[28]	–	Alveo 200	4145	15.8	75.94	18	HLS	0%
ProTEA	Fix8	Alveo U55C	3612	9.12	132	37		
[29]	Fix16	VCU118	5647	2.94	60	11	HLS	93%
ProTEA	Fix8	Alveo U55C	3612	4.48	79	22		0%

comparison. ***ProTEA*** is 0.79× and 6.65× slower than the Intel I5-5257U CPU and JETSON TX2 GPU respectively for model #1 because this study [21] applied a pruning technique. It is 2.5× faster than the NVIDIA TITAN XP GPU for model #2, and 16× faster than the NVIDIA TITAN XP GPU for model #4. These improvements are attributed to higher parallelism, despite ***ProTEA*** operating at a lower frequency and lacking sparsity. For model #3, ***ProTEA*** performed slower than the Intel I5-4460 CPU and NVIDIA RTX 3060 GPU, potentially due to the use of aggressive sparsity and omission of certain computations in the referenced work.

TABLE III
CROSS-PLATFORM COMPARISON

TNNs	Works	Platform	Frequency	Latency (ms)	Speed Up
#1	[21]	INTEL I5-5257U CPU	2.7 GHz	3.54 (Base)	1
		JETSON TX2 GPU	1.3 GHz	0.673	5.3×
		ProTEA FPGA	0.2 GHz	4.48	0.79×
#2	[23]	NVIDIA TITAN XP GPU	1.4 GHz	1.062 (Base)	1
		ProTEA FPGA	0.2 GHz	0.425	2.5×
#3	[25]	INTEL I5-4460 CPU	3.2 GHz	4.66 (Base)	1
		NVIDIA RTX 3060 GPU	1.3 GHz	0.71	6.5×
		ProTEA FPGA	0.2 GHz	5.18	0.89×
#4	[28]	NVIDIA TITAN XP GPU	1.4 GHz	147 (Base)	1
		ProTEA FPGA	0.2 GHz	9.12	16×

VI. CONCLUSION & FUTURE WORKS

In this research, we developed a flexible FPGA-based accelerator for the encoder layer of a transformer neural network (TNN) using a high-level synthesis (HLS) tool. The accelerator architecture exploits FPGA parallelism and the parallel nature of the encoder itself. On the Alveo U55C platform, resources such as BRAMs, DSPs, and LUTs were maximized to enhance parallelism and minimize latency. The accelerator supports runtime programmability, allowing it to adapt to various topologies without requiring re-synthesis. An efficient tiling technique and data loading method for weight matrices were implemented to accommodate large models in on-chip memory, while preventing the overutilization of computational resources. Experimental results show that our design outperforms some CPUs and GPUs in terms of speed and throughput despite operating at a lower frequency and lacking sparsity optimizations. Additionally, it achieved 1.3 to 2.8× speed up compared to the fastest state-of-the-art FPGA-based accelerators. Although this paper focuses solely on encoder layers, future work will extend the architecture to support both encoder and decoder layers of the transformer, using the same design principles.

REFERENCES

[1] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[3] K. Song, K. Wang, H. Yu, Y. Zhang, Z. Huang, W. Luo, X. Duan, and M. Zhang, "Alignment-enhanced transformer for constraining nmt with pre-specified translations," in *AAAI Conference on Artificial Intelligence*, 2020. [Online]. Available: https://api.semanticscholar.org/CorpusID:213842037

[4] T. Wang, L. Gong, C. Wang, Y. Yang, Y. Gao, X. Zhou, and H. Chen, "ViA: A Novel Vision-Transformer Accelerator Based on FPGA," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 11, pp. 4088–4099, Nov. 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9925700/

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[6] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," 2019. [Online]. Available: https://arxiv.org/abs/1907.11692

[7] Z. Liu, P. Yin, and Z. Ren, "An Efficient FPGA-Based Accelerator for Swin Transformer," Aug. 2023, arXiv:2308.13922 [cs]. [Online]. Available: http://arxiv.org/abs/2308.13922

[8] W. Wang, B. Bi, M. Yan, C. Wu, Z. Bao, J. Xia, L. Peng, and L. Si, "Structbert: Incorporating language structures into pre-training for deep language understanding," *arXiv preprint arXiv:1908.04577*, 2019.

[9] L. Lu, Y. Jin, H. Bi, Z. Luo, P. Li, T. Wang, and Y. Liang, "Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 977–991. [Online]. Available: https://doi.org/10.1145/3466752.3480125

[10] T. J. Ham, Y. Lee, S. H. Seo, S. Kim, H. Choi, S. J. Jung, and J. W. Lee, "ELSA: Hardware-Software Co-design for Efficient, Lightweight Self-Attention Mechanism in Neural Networks," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, Jun. 2021, pp. 692–705, iSSN: 2575-713X.

[11] S. Zeng, J. Liu, G. Dai, X. Yang, T. Fu, H. Wang, W. Ma, H. Sun, S. Li, Z. Huang, Y. Dai, J. Li, Z. Wang, R. Zhang, K. Wen, X. Ning, and Y. Wang, "Flightllm: Efficient large language model inference with a complete mapping flow on fpgas," in *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, New York, USA, 2024. [Online]. Available: https://doi.org/10.1145/3626202.3637562

[12] K. Guo, S. Zeng, J. Yu, Y. Wang, and H. Yang, "[dl] a survey of fpga-based neural network inference accelerators," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 12, no. 1, mar 2019. [Online]. Available: https://doi.org/10.1145/3289185

[13] M. Rognlien, Z. Que, J. G. F. Coutinho, and W. Luk, "Hardware-Aware Optimizations for Deep Learning Inference on Edge Devices," in *Applied Reconfigurable Computing. Architectures, Tools, and Applications*, L. Gan, Y. Wang, W. Xue, and T. Chau, Eds. Cham: Springer Nature Switzerland, 2022, vol. 13569, pp. 118–133, series Title: Lecture Notes in Computer Science. [Online]. Available: https://link.springer.com/10.1007/978-3-031-19983-7_9

[14] S. K. Venkataramanaiah, H.-S. Suh, S. Yin, E. Nurvitadhi, A. Dasu, Y. Cao, and J.-S. Seo, "Fpga-based low-batch training accelerator for modern cnns featuring high bandwidth memory," in *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2020, pp. 1–8.

[15] E. Kabir, D. Coble, J. N. Satme, A. R. Downey, J. D. Bakos, D. Andrews, and M. Huang, "Accelerating lstm-based high-rate dynamic system models," in *2023 33rd International Conference on Field-Programmable Logic and Applications (FPL)*, 2023, pp. 327–332.

[16] B. Zhang, H. Zeng, and V. Prasanna, "Accelerating large scale gcn inference on fpga," in *2020 IEEE 28th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2020, pp. 241–241.

[17] Z. Que, M. Loo, H. Fan, M. Pierini, A. Tapper, and W. Luk, "Optimizing Graph Neural Networks for Jet Tagging in Particle Physics on FPGAs," in *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*. Belfast, United Kingdom: IEEE, Aug. 2022, pp. 327–333. [Online]. Available: https://ieeexplore.ieee.org/document/10035216/

[18] S. Lu, M. Wang, S. Liang, J. Lin, and Z. Wang, "Hardware Accelerator for Multi-Head Attention and Position-Wise Feed-Forward in the Transformer," in *2020 IEEE 33rd International System-on-Chip Conference (SOCC)*. Las Vegas, NV, USA: IEEE, Sep. 2020, pp. 84–89. [Online]. Available: https://ieeexplore.ieee.org/document/9524802/

[19] T. J. Ham, S. Jung, S. Kim, Y. H. Oh, Y. Park, Y. Song, J.-H. Park, S. Lee, K. Park, J. W. Lee, and D.-K. Jeong, "A3: Accelerating attention mechanisms in neural networks with approximation," *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 328–341, 2020. [Online]. Available: https://api.semanticscholar.org/CorpusID:211296403

[20] H. Peng, S. Huang, S. Chen, B. Li, T. Geng, A. Li, W. Jiang, W. Wen, J. Bi, H. Liu, and C. Ding, "A length adaptive algorithm-hardware co-design of transformer on FPGA through sparse attention and dynamic pipelining," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*. San Francisco California: ACM, Jul. 2022, pp. 1135–1140. [Online]. Available: https://dl.acm.org/doi/10.1145/3489517.3530585

[21] H. Peng, S. Huang, T. Geng, A. Li, W. Jiang, H. Liu, S. Wang, and C. Ding, "Accelerating Transformer-based Deep Learning Models on FPGAs using Column Balanced Block Pruning," in *2021 22nd International Symposium on Quality Electronic Design (ISQED)*. Santa Clara, CA, USA: IEEE, Apr. 2021, pp. 142–148. [Online]. Available: https://ieeexplore.ieee.org/document/9424344/

[22] Z. Jiang, D. Yin, E. E. Khoda, V. Loncar, E. Govorkova, E. Moreno, P. Harris, S. Hauck, and S.-C. Hsu, "Ultra Fast Transformers on FPGAs for Particle Physics Experiments."

[23] F. Wojcicki, Z. Que, A. D. Tapper, and W. Luk, "Accelerating Transformer Neural Networks on FPGAs for High Energy Physics Experiments," in *2022 International Conference on Field-Programmable Technology (ICFPT)*. Hong Kong: IEEE, Dec. 2022, pp. 1–8. [Online]. Available: https://ieeexplore.ieee.org/document/9974463/

[24] Y. Chen, T. Li, X. Chen, Z. Cai, and T. Su, "High-Frequency Systolic Array-Based Transformer Accelerator on Field Programmable Gate Arrays," *Electronics*, vol. 12, no. 4, p. 822, Jan. 2023, number: 4 Publisher: Multidisciplinary Digital Publishing Institute. [Online]. Available: https://www.mdpi.com/2079-9292/12/4/822

[25] X. Yang and T. Su, "EFA-Trans: An Efficient and Flexible Acceleration Architecture for Transformers," *Electronics*, vol. 11, no. 21, p. 3550, Oct. 2022. [Online]. Available: https://www.mdpi.com/2079-9292/11/21/3550

[26] Y. Bai and F. University, "LTrans-OPU: A Low-Latency FPGA-Based Overlay Processor for Transformer Networks."

[27] P. Ganesh, Y. Chen, X. Lou, M. A. Khan, Y. Yang, H. Sajjad, P. Nakov, D. Chen, and M. Winslett, "Compressing large-scale transformer-based models: A case study on bert," *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 1061–1080, 2021.

[28] P. Qi, E. H.-M. Sha, Q. Zhuge, H. Peng, S. Huang, Z. Kong, Y. Song, and B. Li, "Accelerating Framework of Transformer by Hardware Design and Model Compression Co-Optimization," in *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. Munich, Germany: IEEE, Nov. 2021, pp. 1–9. [Online]. Available: https://ieeexplore.ieee.org/document/9643586/

[29] B. Li, S. Pandey, H. Fang, Y. Lyv, J. Li, J. Chen, M. Xie, L. Wan, H. Liu, and C. Ding, "FTRANS: energy-efficient acceleration of transformers using FPGA," in *Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design*. Boston Massachusetts: ACM, Aug. 2020, pp. 175–180. [Online]. Available: https://dl.acm.org/doi/10.1145/3370748.3406567

[30] Z. Luo, L. Lu, Y. Jin, L. Jia, and Y. Liang, "Calabash: Accelerating Attention Using a Systolic Array Chain on FPGAs," in *2023 33rd International Conference on Field-Programmable Logic and Applications (FPL)*. Gothenburg, Sweden: IEEE, Sep. 2023, pp. 242–247. [Online]. Available: https://ieeexplore.ieee.org/document/10296242/

[31] H. Wang, Z. Zhang, and S. Han, "Spatten: Efficient sparse attention architecture with cascade token and head pruning," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 97–110.

[32] R. Qiao, X. Guo, W. Mao, J. Li, and H. Lu, "FPGA-based design and implementation of the location attention mechanism in neural networks," *Journal of Intelligent & Fuzzy Systems*, vol. 43, no. 4, pp. 5309–5323, Aug. 2022. [Online]. Available: https://www.medra.org/servlet/aliasResolver?alias=iospress&doi=10.3233/JIFS-212273

[33] X. Zhang, Y. Wu, P. Zhou, X. Tang, and J. Hu, "Algorithm-hardware Co-design of Attention Mechanism on FPGA Devices," *ACM Transactions on Embedded Computing Systems*, vol. 20, no. 5s, pp. 1–24, Oct. 2021. [Online]. Available: https://dl.acm.org/doi/10.1145/3477002

[34] "AMD Technical Information Portal — docs.amd.com," https://docs.amd.com/r/en-US/ug1399-vitis-hls/AXI4-Master-Interface, [Accessed 21-07-2024].

[35] "AMD Technical Information Portal — docs.amd.com," https://docs.amd.com/r/en-US/ug1399-vitis-hls/AXI4-Lite-Interface, [Accessed 10-08-2024].

[36] "MicroBlaze Processor Reference Guide," 2022. [Online]. Available: https://docs.xilinx.com/v/u/en-US/ug984-vivado-microblaze-ref